

ROCA • New Member Onboarding • Design Document

Field	Value
Document	New Member Onboarding — Design Document
Version	1.0
Status	Ready for Development
Audience	Engineering Team
Tech Stack	React 18 · Node.js · PostgreSQL · Prisma · JWT

1. Overview & Purpose

This document specifies the complete design and implementation details for the New Member Onboarding user flow in the ROCA (Religious Organization Calendar Application) platform. Onboarding is the single most important first impression a new community member will have of the platform — it must feel warm, personal, and spiritually grounded rather than transactional.

The onboarding flow guides a new member from initial sign-up through profile completion, organization selection, interest configuration, notification preferences, and an interactive feature tour — all in a 7-step progressive wizard that takes approximately 5–8 minutes to complete.

1.1 Design Principles

The following principles govern every design decision in this flow:

- Warm first, functional second** — use church-appropriate language, imagery, and tone throughout
- Progressive disclosure** — only ask for information at the moment it's needed; never overwhelm
- Mobile-first** — the majority of new members will encounter this flow on a smartphone
- Skip-able, resumable** — members can skip optional steps and complete them later; progress is never lost
- Spiritually contextual** — the liturgical season banner and scripture card appear from Step 1, establishing the ROCA personality immediately

1.2 Success Metrics

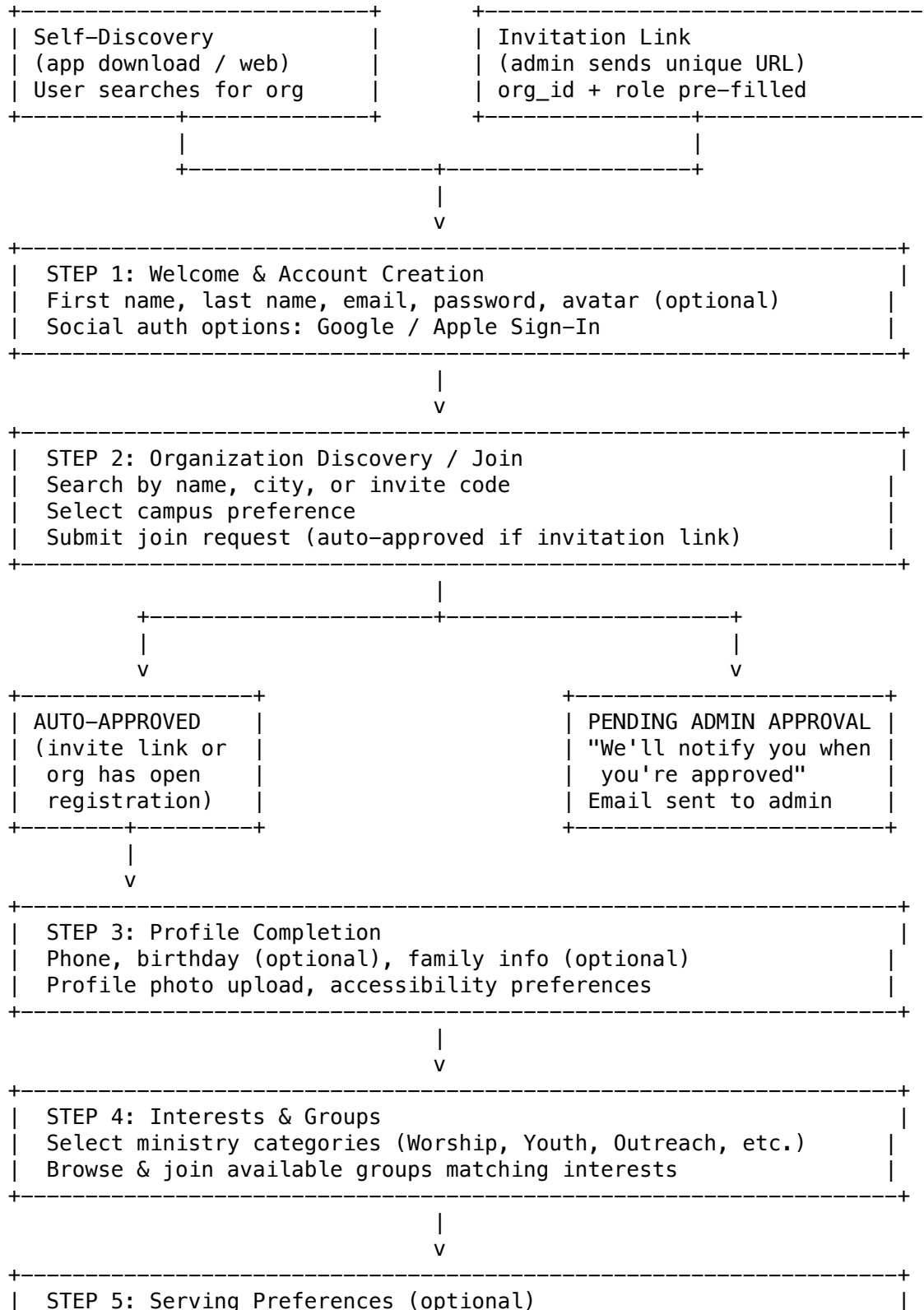
Metric	Target	Measurement
Wizard completion rate	≥ 75% (within 7 days)	Analytics funnel
Step 1–2 drop-off	< 10%	Per-step completion events
Time to first event RSVP	≤ 72 hours post-onboard	Event registration timestamp
Time to first volunteer sign-up	≤ 14 days post-onboard	Volunteer assignment timestamp
Group join rate	≥ 50% within 30 days	group_members table count
Notification opt-in rate	≥ 80%	Notification preferences record
Admin approval time	< 24 hours (median)	approved_at - registered_at

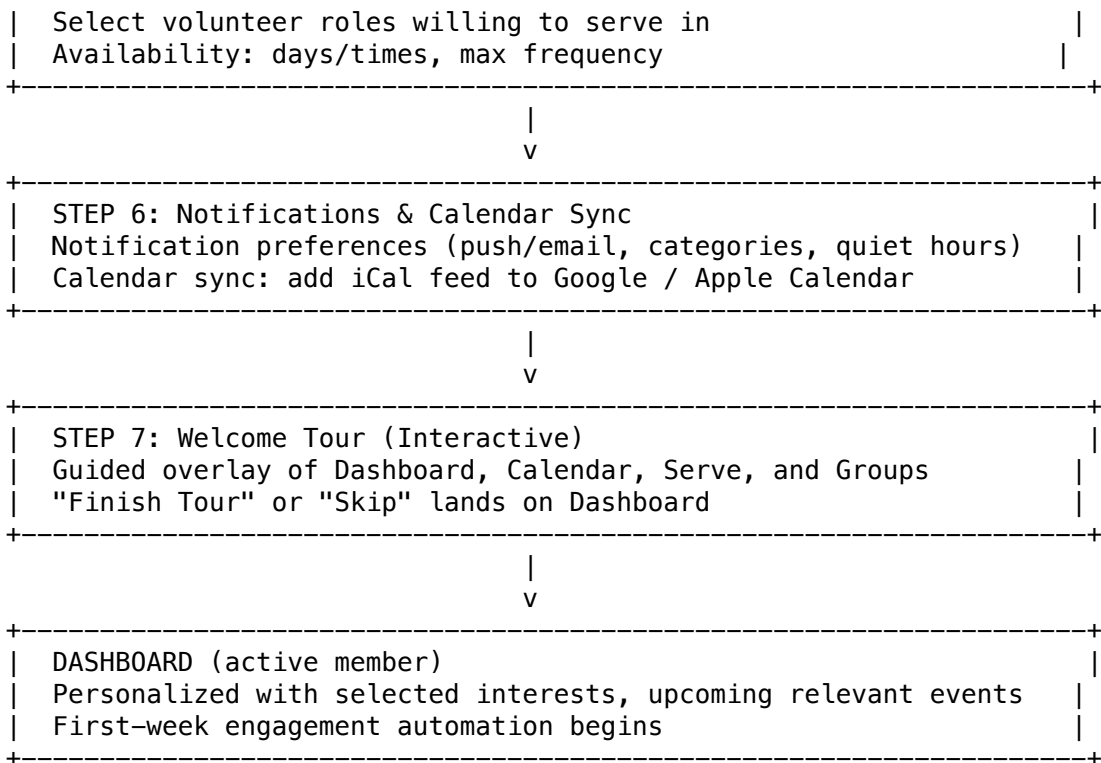
2. User Flow & State Diagram

2.1 High-Level Onboarding Flow

The onboarding flow has two entry paths — self-discovery registration and admin-generated invitation links — and a shared 7-step wizard that follows account creation.

ENTRY POINTS

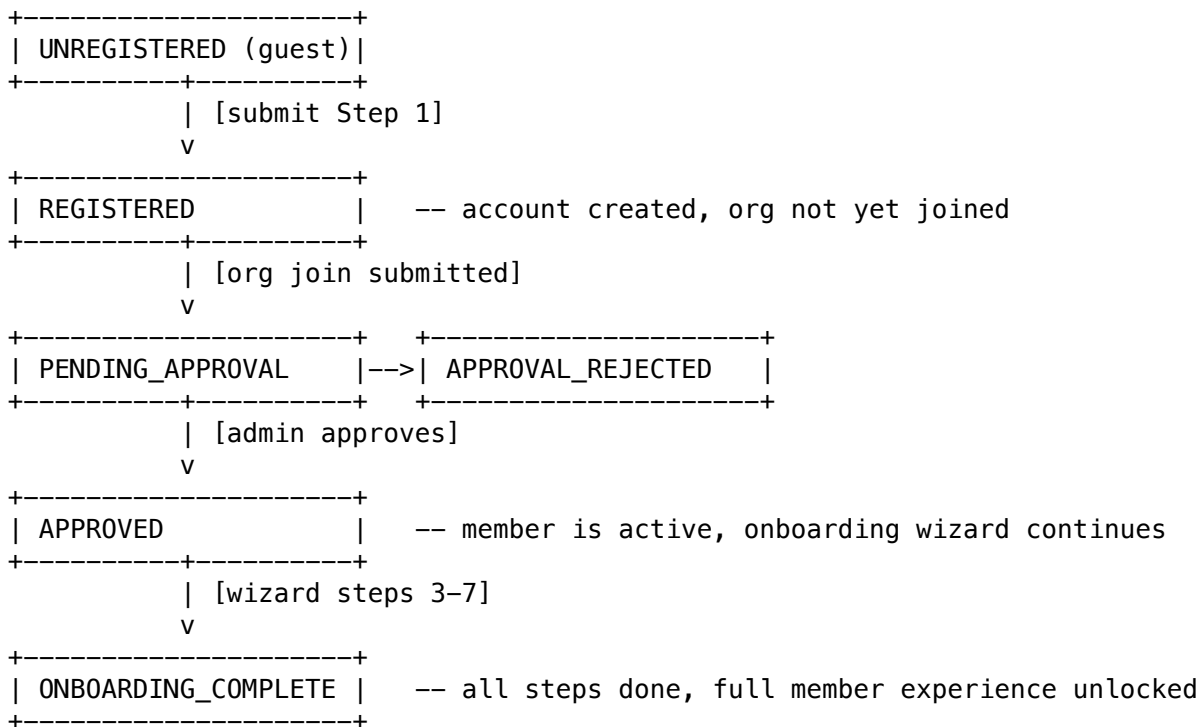




2.2 Onboarding State Machine

Each onboarding session is tracked via the `onboarding_status` field on the members table, enabling resume-on-return behavior:

STATES:



Note: APPROVED members who skip steps land in ONBOARDING_INCOMPLETE. They can complete remaining steps any time from Settings > Complete Profile.

3. Screen Wireframes & Descriptions

All 7 onboarding web screens share a common layout: a centered card (680px max-width) on a lightly tinted background that reflects the current liturgical season color. A liturgical season banner appears above the card. The ROCA wordmark sits in the top-left. A progress indicator (pill-style steps 1–7) is fixed at the top of the card.

Screen 1 — Welcome & Account Creation

ROCA	Already a member? [Sign In]
[Liturgical Season Banner: Ordinary Time – Week 21]	
Welcome to Grace Community Church Create your account to get started	
[Step 1 of 7] [Step 2] [Step 3] [Step 4] [5] [6] [7]	
First Name *	Last Name *
Sarah	Johnson
Email Address *	
sarah.johnson@email.com	
Password * (Show/Hide)	
.....	
[Strength: Strong ===] Min 8 chars, 1 uppercase	
Profile Photo (optional) [Upload Photo] or [Take Photo]	
---- OR ----	
[Continue with Google] [Continue with Apple]	
By creating an account, you agree to our Terms of Service and Privacy Policy.	
[Create Account ->]	

Key Design Notes:

1. Password strength meter uses zxcvbn scoring; displayed as colored progress bar
2. Social auth (Google/Apple) bypasses email/password fields; still captures first/last name via OAuth profile
3. Profile photo upload is optional and can be skipped; defaults to initials-based avatar
4. Organization context appears in the header if user arrived via an invite link

Screen 2 — Organization Discovery & Join

[Step 1 Complete v] [Step 2 of 7] [3][4][5][6][7]					
Find Your Church Community					
Search for your organization:					
[Search icon] Grace Community	[x]				
<table border="1"> <tr> <td>Grace Community Church</td> <td>Main St, Austin TX</td> </tr> <tr> <td>3 campuses 2,400 members</td> <td>[Select ->]</td> </tr> </table>		Grace Community Church	Main St, Austin TX	3 campuses 2,400 members	[Select ->]
Grace Community Church	Main St, Austin TX				
3 campuses 2,400 members	[Select ->]				
<table border="1"> <tr> <td>Grace Chapel</td> <td>Denver, CO</td> </tr> <tr> <td>1 campus 850 members</td> <td>[Select ->]</td> </tr> </table>		Grace Chapel	Denver, CO	1 campus 850 members	[Select ->]
Grace Chapel	Denver, CO				
1 campus 850 members	[Select ->]				
---- OR have an invite code? ----					
Enter invite code...					
[Redeem Code]					
--- After selecting an org ---					
Select your primary campus:					
(o) Main Campus - 123 Church St, Austin TX					
() North Campus - 456 Oak Ave, Austin TX					
() Online Campus					
[<- Back]	[Request to Join ->]				

Key Design Notes:

1. Search hits GET /organizations?q= with debounce (300ms)
2. Invite code input auto-validates against the invitation_codes table on blur
3. Invite code auto-fills organization and campus; search field becomes read-only
4. Join request auto-approves if org has open_registration = true or invite code is valid

Screen 3 — Pending Approval (Conditional)

[Hourglass illustration]
You're on your way!
Your request to join Grace Community Church has been submitted to the admin team.
You'll receive an email at sarah@email.com when your account is approved (usually within 24h).
In the meantime, explore public events:

+-----+	
Sunday Worship	Aug 24 - 9:00 AM
Women's Bible Study	Aug 28 - 7:00 PM
Fall Festival	Sep 28 - 4:00 PM
+-----+	
[View Public Calendar]	[Resend confirmation email]
+-----+	

Screen 4 — Profile Completion

+-----+	
[1v] [2v] [3v] [Step 4 of 7] [5] [6] [7]	
Tell us a little about yourself	
Phone Number (optional) Date of Birth (optional)	
+-----+ +-----+	
(512) 555-0100	MM / DD / YYYY
+-----+	
Family / Household (optional)	
[+ Add family member]	
-- Sarah Johnson (self)	
-- [+ Spouse/Partner name]	
-- [+ Child name, grade]	
Accessibility Preferences (optional)	
[x] Show wheelchair-accessible events	
[x] Show childcare availability	
[] Require livestream option	
Preferred contact method:	
(o) Email () SMS () Both	
[<- Back] [Skip for now] [Continue ->]	
+-----+	

Screen 5 — Interests & Group Discovery

+-----+			
[1v] [2v] [3v] [4v] [Step 5 of 7] [6] [7]			
What matters most to you?			
Select all that apply			
+-----+	+-----+	+-----+	+-----+
[Music]	[Youth]	[Serv.]	[Outreach]
Worship	Families	Serve &	Community
& Music	& Kids	Volunteer	Outreach
[SELECT]	[SELECT]	[SELECT]	[SELECT]
+-----+			
+-----+	+-----+	+-----+	+-----+
[Prayer]	[Study]	[Social]	[Admin]
Prayer &	Bible &	Fellowship	Leadership
Spiritual	Teaching	& Social	& Admin
[SELECT]	[SELECT]	[SELECT]	[SELECT]
+-----+			

Groups matching your interests:		
+-----+		
Women's Bible Study	Thu 7PM	Fellowship Hall
14 members	Led by Sarah K.	[+ Join Group]
+-----+		
+-----+		
Worship Team	Sun 8AM warmup	Sanctuary
8 members	Led by Mike R.	[+ Join Group]
+-----+		
+-----+		
[<- Back]	[Skip for now]	[Continue ->]

Screen 6 — Serving & Volunteer Preferences

[1v] [2v] [3v] [4v] [5v] [Step 6 of 7] [7]		
How would you like to serve?		
This is optional and can be changed any time		
Volunteer roles I'm interested in:		
☒ Greeter / Welcome Team		
☐ Usher		
☐ Sound / AV Tech		
☐ Worship Team (instrument / voice)		
☐ Nursery & Children's Ministry		
☐ Welcome Desk / Info Table		
☐ Community Outreach / Food Pantry		
My typical availability:		
Days: ☒ Sunday ☐ Mon ☐ Tue ☒ Wed etc.		
Sessions: ☒ Morning ☐ Afternoon ☒ Evening		
How often can you serve?		
☐ Weekly ☐ Every 2 weeks ☐ Monthly		
[<- Back] [Skip Serving] [Continue ->]		

Screen 7 — Notifications & Calendar Sync

[1v] [2v] [3v] [4v] [5v] [6v] [Step 7 of 7]		
Stay connected with your community		
Notify me about:		
☒ Upcoming events I'm registered for		
☒ My volunteer assignments & reminders		
☒ New events in my interest categories		
☒ Group announcements		
☒ Volunteer openings matching my skills		
☐ Weekly prayer & scripture digest		
Notification method:		
☐ Push + Email ☐ Push only ☐ Email only		

```
|
| Quiet hours (no notifications):
| From [10:00 PM] To [7:00 AM]
|
| Sync with your calendar app:
| [Add to Google Calendar] [Add to Apple Calendar]
| [Copy iCal Link]
|
| [← Back] [Finish Setup →]
|
+-----+
```

Mobile Screen Layouts

On mobile, the onboarding wizard uses the full screen with a sticky progress bar at top and sticky action buttons at the bottom. The keyboard automatically scrolls the active field into view. Steps 5 (Serving) and 6 (Notifications) are collapsed into a single scrollable screen on mobile to reduce total steps.

Mobile Step 1	Mobile Step 5 (Interests)	Mobile Step 6 (Notifs)
+-----+	+-----+	+-----+
[Status bar]	[Status bar]	[Status bar]
ROCA [X]	<- Back 5 of 7	<- Back 6 of 7
--- Progress ---	--- Progress ---	--- Progress ---
Welcome to	What matters	Stay connected
Grace Community	most to you?	
First Name	+-----+ +-----+	Notify me about:
+-----+	Worshp Youth	[ON] Events
Sarah	[SEL] [SEL]	[ON] Volunteer
+-----+	+-----+ +-----+	[ON] Groups
Last Name	+-----+ +-----+	[OFF] Digest
+-----+	Serve Outchr	Method:
Johnson	[SEL] [SEL]	(o)Push+Email
+-----+	+-----+ +-----+	Calendar:
Email	Matched groups:	[Google Calendar]
+-----+	Women's Bible	[Apple Calendar]
sarah@...	Study Thu 7PM	
+-----+	[+ Join]	
[Create Account →]	[Skip] [Continue→]	[Finish Setup →]
+-----+	+-----+	+-----+

4. User Stories & EARS Acceptance Criteria

The following 10 user stories cover the complete onboarding flow. Acceptance criteria follow the EARS (Easy Approach to Requirements Syntax) notation: WHEN [trigger], THE SYSTEM SHALL [behavior].

US-1: Self-Registration with Email or Social Auth

As a prospective community member, **I want to** create an account using my email or existing social account, **so that** I can join my church's digital community without friction.

1. WHEN a user submits a valid first name, last name, email, and password, THE SYSTEM SHALL create a members record with status REGISTERED and return a JWT access/refresh token pair.
2. WHEN the submitted email already exists in the members table for the same org, THE SYSTEM SHALL return error AUTH_004 ("Email already registered") and prompt the user to sign in.
3. WHEN a user authenticates via Google or Apple OAuth, THE SYSTEM SHALL upsert the member record using the OAuth sub as the identity key and skip password requirements.
4. WHEN a user submits a password shorter than 8 characters or missing an uppercase letter, THE SYSTEM SHALL display inline validation before form submission.
5. WHEN account creation succeeds, THE SYSTEM SHALL send a welcome email containing the church's name, logo, and a "Verify Email" link within 60 seconds.

US-2: Invitation-Based Registration

As a new member invited by a church admin, **I want to** register via a personal invite link, **so that** my org and role are pre-filled and I don't need admin approval.

1. WHEN a user arrives via a valid invitation URL (/invite/{code}), THE SYSTEM SHALL pre-fill the org_id, campus_id, and role from the invitation_codes table and skip org search.
2. WHEN a user arrives via an expired invitation link (TTL elapsed), THE SYSTEM SHALL display an error: "This invitation link has expired. Please contact your admin for a new one."
3. WHEN a user redeems a single-use invitation code, THE SYSTEM SHALL mark the code as used_at and prevent re-use by any other user.
4. WHEN a user redeems a multi-use invitation code, THE SYSTEM SHALL increment the uses_count and auto-approve the member without admin review.
5. WHEN an invitation specifies a role (e.g., leader), THE SYSTEM SHALL assign that role to the new member upon approval, overriding the default member role.

US-3: Organization Discovery & Join

As a new user without an invite link, **I want to** search for and join my church, **so that** I can connect my account to the correct organization and campus.

1. WHEN a user types 2+ characters in the org search field, THE SYSTEM SHALL query GET /organizations?q= and display results within 500ms.
2. WHEN a user selects an organization, THE SYSTEM SHALL display all campuses for that org and require campus selection before proceeding.
3. WHEN a user submits a join request and the org has open_registration = true, THE SYSTEM SHALL auto-approve and advance to Step 3 without admin intervention.
4. WHEN a user submits a join request and the org requires admin approval, THE SYSTEM SHALL create a join request with status PENDING and send a notification to all org admins.
5. WHEN a user enters a valid invite code in the search screen, THE SYSTEM SHALL automatically select the org and campus associated with that code.

US-4: Admin Approval Workflow

As a church administrator, **I want to** review and approve new member join requests, **so that** I maintain control over who joins my organization's digital community.

1. WHEN a new join request is submitted, THE SYSTEM SHALL display a notification badge on the admin's Members dashboard and send an email within 2 minutes.

2. WHEN an admin approves a join request, THE SYSTEM SHALL update the member's status to APPROVED, send a push + email notification to the member, and include a deep link to resume onboarding.
3. WHEN an admin rejects a join request with a reason, THE SYSTEM SHALL set status to APPROVAL_REJECTED and email the rejected user with the provided reason.
4. WHEN a join request remains PENDING for more than 48 hours, THE SYSTEM SHALL send a reminder notification to all org admins.

US-5: Profile Completion

As a newly approved member, **I want to** complete my profile with contact info and family details, **so that** the church can reach me and I receive relevant event recommendations.

1. WHEN a user uploads a profile photo, THE SYSTEM SHALL resize to 400×400px, convert to JPEG, and store in S3 under /avatars/{member_id}.jpg.
2. WHEN a user adds a family member with a child's name and grade, THE SYSTEM SHALL flag the member as has_children = true for childcare-relevant event filtering.
3. WHEN accessibility preferences are saved, THE SYSTEM SHALL apply those filters as defaults to all calendar and event discovery views.
4. WHEN a user skips Step 3, THE SYSTEM SHALL set the onboarding_status to ONBOARDING_INCOMPLETE and display a "Complete your profile" nudge in the dashboard header for 14 days.

US-6: Interest Selection & Group Matching

As a new member, **I want to** select my ministry interests and see matching groups, **so that** my calendar feed is immediately relevant and I feel connected.

1. WHEN a user selects one or more interest categories, THE SYSTEM SHALL save them as member_interests records and use them to filter upcoming events on the dashboard.
2. WHEN a user selects interests, THE SYSTEM SHALL query GET /groups?interests=[] and display matching groups sorted by campus proximity.
3. WHEN a user joins a group from the discovery screen, THE SYSTEM SHALL create a group_members record and send the group leader a notification.
4. WHEN no groups match the selected interests, THE SYSTEM SHALL display "No groups found yet — check back soon" without an error state.

US-7: Serving & Volunteer Preferences

As a new member willing to volunteer, **I want to** indicate my serving preferences upfront, **so that** the auto-scheduler can include me when gaps arise in my preferred roles.

1. WHEN a user selects one or more volunteer roles, THE SYSTEM SHALL create member_volunteer_preferences records linking member_id to each selected role_id.
2. WHEN a user sets availability (days/times), THE SYSTEM SHALL store the schedule in a JSONB field on the member record for use by the auto-scheduler.
3. WHEN a user sets maximum serving frequency (weekly / bi-weekly / monthly), THE SYSTEM SHALL respect this constraint in all auto-scheduling runs.
4. WHEN this step is skipped, THE SYSTEM SHALL not add the member to any volunteer rotation and shall not surface volunteer gaps in their notifications.

US-8: Notification & Calendar Preferences

As a new member completing onboarding, **I want to** configure how and when I receive notifications, **so that** I stay informed without being overwhelmed.

1. WHEN a user sets notification preferences, THE SYSTEM SHALL create or update a `notification_preferences` record with per-category opt-in flags and quiet hours.
2. WHEN a user requests push notifications (mobile), THE SYSTEM SHALL trigger the native OS permission dialog before registering the device token.
3. WHEN a user clicks "Add to Google Calendar", THE SYSTEM SHALL generate an authenticated iCal subscription URL and open the Google Calendar add-subscription deep link.
4. WHEN a user clicks "Copy iCal Link", THE SYSTEM SHALL copy the subscription URL to clipboard and display a "Copied!" confirmation toast.

US-9: Skip & Resume Onboarding

As a new member short on time, **I want to** skip optional steps and return later, **so that** I can access the app immediately without friction.

1. WHEN a user clicks "Skip for now" on Steps 3, 4, 5, or 6, THE SYSTEM SHALL advance to the next step and persist the current progress in the `onboarding_step` column.
2. WHEN an `ONBOARDING_INCOMPLETE` member logs in on a new session, THE SYSTEM SHALL display a resume banner: "Complete your profile — you're X steps away" with a "Continue" CTA.
3. WHEN a member completes all steps including previously skipped ones, THE SYSTEM SHALL update `onboarding_status` to `ONBOARDING_COMPLETE` and remove the completion nudge.
4. WHEN the browser or app is closed mid-flow (after Step 1), THE SYSTEM SHALL persist all completed step data so the user resumes at the last incomplete step.

US-10: First-Week Engagement Automation

As a church admin, **I want** newly onboarded members to receive curated first-week touchpoints, **so that** they feel welcomed and engage with church activities quickly.

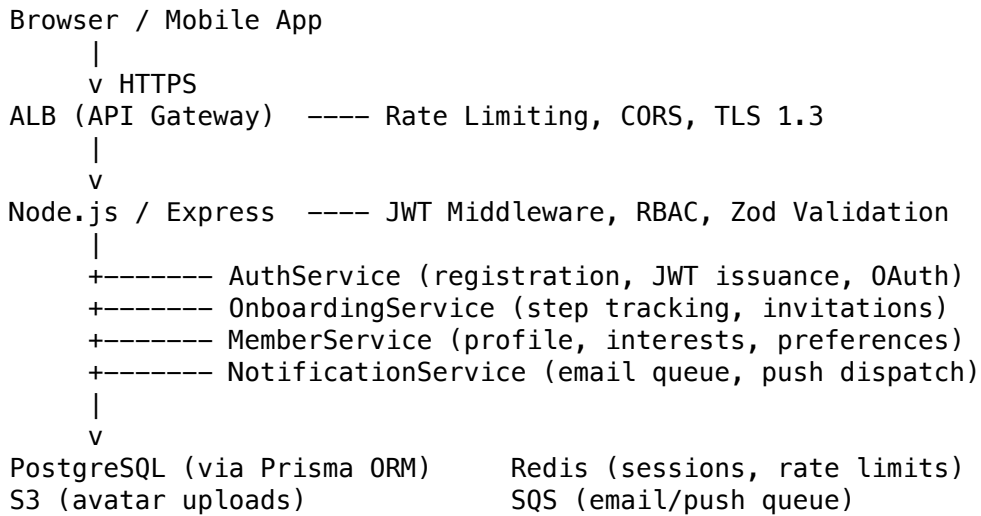
1. WHEN a member achieves `ONBOARDING_COMPLETE` status, THE SYSTEM SHALL enqueue a 3-message engagement sequence: Day 1 (welcome), Day 3 (upcoming events), Day 7 (groups & serving).
2. WHEN the Day-1 welcome message is sent, THE SYSTEM SHALL personalize it with the member's first name, selected campus, and the next upcoming worship service.
3. WHEN the Day-3 message is sent, THE SYSTEM SHALL include 3 upcoming events matching the member's selected interest categories.
4. WHEN the Day-7 message is sent and the member has not yet joined a group, THE SYSTEM SHALL include 2 group recommendations based on their interest profile.
5. WHEN a member has already joined a group before Day 7, THE SYSTEM SHALL substitute a volunteer opportunities message instead.

5. Implementation Architecture

This section describes the full technical implementation tied to the ROCA tech stack: React 18, Node.js /

Express, PostgreSQL, Prisma ORM, and JWT-based authentication. All endpoints follow the existing /api/v1/ base URL convention.

5.1 Architecture Overview



5.2 Database Schema Changes

The following additions to the existing ROCA schema support the onboarding flow. All existing tables remain unchanged; new tables and columns are additive.

New Columns on members Table:

Column	Type	Default	Purpose
onboarding_status	VARCHAR(30)	REGISTERED	Tracks wizard completion stage
onboarding_step	INTEGER	1	Last completed wizard step (1-7)
invite_code_used	UUID NULL	NULL	FK to invitation_codes.code_id
has_children	BOOLEAN	false	Enables childcare event filtering
accessibility_prefs	JSONB	{}	Wheelchair, childcare, livestream flags
availability_schedule	JSONB	{}	Days/times available for volunteering
serve_max_frequency	VARCHAR(20)	biweekly	Volunteer frequency constraint
approved_at	TIMESTAMPTZ	NULL	When admin approval was granted
email_verified_at	TIMESTAMPTZ	NULL	When email verification link was clicked

New Tables:

```
-- invitation_codes - Stores admin-generated invite tokens
CREATE TABLE invitation_codes (
  code_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id           UUID NOT NULL REFERENCES organizations(org_id),
  campus_id        UUID REFERENCES campuses(campus_id),
  role             VARCHAR(20) DEFAULT 'member',
  code             VARCHAR(32) UNIQUE NOT NULL, -- 8-char alphanumeric
  is_multi_use     BOOLEAN NOT NULL DEFAULT false,
  max_uses         INTEGER, -- NULL = unlimited
  uses_count       INTEGER NOT NULL DEFAULT 0,
```

```

    expires_at      TIMESTAMPTZ,
    created_by      UUID REFERENCES members(member_id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    used_at         TIMESTAMPTZ -- for single-use codes
);

CREATE INDEX idx_invite_codes_code ON invitation_codes(code);
CREATE INDEX idx_invite_codes_org ON invitation_codes(org_id, expires_at);

-- member_interests - Many-to-many: member ↔ interest_category
CREATE TABLE member_interests (
    member_id       UUID NOT NULL REFERENCES members(member_id),
    category_id     UUID NOT NULL REFERENCES event_categories(category_id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (member_id, category_id)
);

-- member_volunteer_preferences - Many-to-many: member ↔ volunteer_role
CREATE TABLE member_volunteer_preferences (
    member_id       UUID NOT NULL REFERENCES members(member_id),
    role_id         UUID NOT NULL REFERENCES volunteer_roles(role_id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (member_id, role_id)
);

-- notification_preferences - Per-member notification configuration
CREATE TABLE notification_preferences (
    pref_id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    member_id       UUID UNIQUE NOT NULL REFERENCES members(member_id),
    channel         VARCHAR(10) NOT NULL DEFAULT 'both', -- push|email|both
    events_opted_in BOOLEAN NOT NULL DEFAULT true,
    volunteer_opted_in BOOLEAN NOT NULL DEFAULT true,
    groups_opted_in BOOLEAN NOT NULL DEFAULT true,
    digest_opted_in BOOLEAN NOT NULL DEFAULT false,
    quiet_from      TIME DEFAULT '22:00:00',
    quiet_to        TIME DEFAULT '07:00:00',
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

5.3 API Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	None	Create account; returns JWT pair
POST	/auth/register/social	None	OAuth registration (Google/Apple)
POST	/auth/verify-email	Token link	Verify email address via tokenized link
GET	/organizations?q=	None	Search orgs by name/city for discovery
GET	/invitations/:code	None	Validate invite code; returns org+campus+role
POST	/organizations/:id/join	JWT	Submit org join request
PUT	/members/me/profile	JWT	Update profile (Step 3 data)
PUT	/members/me/interests	JWT	Save interest category selections
PUT	/members/me/volunteer-prefs	JWT	Save volunteer role & availability prefs
PUT	/members/me/notification-prefs	JWT	Save notification channel & category prefs
POST	/members/me/avatar	JWT	Upload profile photo (multipart)

Method	Endpoint	Auth	Description
GET	/members/me/onboarding-status	JWT	Returns current step + completion status
PUT	/members/me/onboarding-step	JWT	Advance or record last completed step
POST	/admin/invitations	admin+	Generate a new invitation code
GET	/admin/join-requests	admin+	List pending member join requests
PUT	/admin/join-requests/:id	admin+	Approve or reject a join request
GET	/calendar/ical/subscription	JWT	Generate authenticated iCal subscription URL

5.4 Frontend Component Architecture (React 18)

The onboarding wizard is a single React component tree mounted at /onboarding. It uses Zustand for multi-step form state persistence and React Query for API calls. The wizard is lazy-loaded and code-split from the main app bundle.

```
<OnboardingRoute>
  <LiturgicalBanner />           // shared with main app, shows season+color
  <OnboardingWizard>
    <WizardProgressBar steps={7} current={step} />
    {step === 1 && <StepAccountCreation />}
    {step === 2 && <StepOrgDiscovery />}
    {step === 2.5 && <StepPendingApproval />} // conditional
    {step === 3 && <StepProfileCompletion />}
    {step === 4 && <StepInterestsGroups />}
    {step === 5 && <StepServingPreferences />}
    {step === 6 && <StepNotificationsCalendar />}
    {step === 7 && <StepWelcomeTour />}
    <WizardNavigation
      onBack={handleBack}
      onNext={handleNext}
      onSkip={handleSkip}
      isLoading={mutation.isPending}
    />
  </OnboardingWizard>
</OnboardingRoute>
```

Zustand Store: useOnboardingStore

```
interface OnboardingStore {
  currentStep: number;
  stepData: {
    step1: AccountFormData | null;
    step2: OrgSelectionData | null;
    step3: ProfileFormData | null;
    step4: InterestSelectionData | null;
    step5: ServingPrefsData | null;
    step6: NotificationPrefsData | null;
  };
  member: MemberProfile | null;
  setStep: (step: number) => void;
  setStepData: <K extends keyof StepData>(key: K, data: StepData[K]) => void;
  completedSteps: Set<number>;
  markStepComplete: (step: number) => void;
  reset: () => void;
}
```

5.5 Mobile Implementation Notes (React Native / Expo)

1. The onboarding wizard uses a Pager view (`react-native-pager-view`) for smooth horizontal swipe between steps.
2. Push notification permission prompt is triggered at Step 6 (Notifications) using Expo Notifications API. Permission must be requested before token registration.
3. Biometric lock setup is offered at Step 7 completion via Expo `LocalAuthentication`; stored in Expo `SecureStore`.
4. Profile photo upload uses Expo `ImagePicker` with client-side resizing (max 800x800) before upload to S3 presigned URL.
5. Steps are persisted to `AsyncStorage` after each advance so the flow can resume after app backgrounding.

6. Technical Specifications

6.1 Invitation Code System

Property	Specification
Code format	8-character alphanumeric, uppercase, no ambiguous chars (0, O, I, 1)
Generation	<code>crypto.randomBytes(6).toString('base64url').toUpperCase().slice(0,8)</code>
Default TTL	30 days from creation; admin-configurable at generation time
Single-use	<code>used_at</code> set on first redemption; subsequent attempts return <code>INVITE_001</code> error
Multi-use	<code>uses_count</code> incremented on each redemption; <code>max_uses = NULL</code> for unlimited
URL format	<code>https://roca.app/invite/{code}</code> or <code>https://yourchurch.roca.app/join/{code}</code>
Error codes	<code>INVITE_001</code> : Code not found; <code>INVITE_002</code> : Expired; <code>INVITE_003</code> : Max uses reached

6.2 Engagement Automation (Post-Onboarding Email Sequence)

Triggered by a Lambda function subscribed to the `member.onboarding_completed` SQS event. Each message is personalized and respects the member's notification preferences.

Day	Message Type	Content & Personalization
Day 1	Welcome	"{FirstName}, welcome to {ChurchName}!" + next worship service + link to dashboard
Day 3	Discover Events	3 upcoming events matching member interests + RSVP deep links
Day 7A	Groups (no group)	Sent if member hasn't joined a group: 2 group recommendations + join links
Day 7B	Volunteer (has group)	Sent if member joined a group: volunteer opening opportunities matching their prefs

6.3 Interest-to-Event Personalization

After onboarding, the dashboard's Upcoming Events feed applies a scoring function to rank events by relevance:

-- PostgreSQL query for personalized event feed

```
SELECT e.*,
CASE
  WHEN mi.category_id IS NOT NULL THEN 2 -- matches member interest
  WHEN e.campus_id = m.campus_id THEN 1 -- same campus
  ELSE 0
END AS relevance_score
FROM events e
LEFT JOIN member_interests mi
  ON mi.member_id = $memberId
  AND mi.category_id = e.category_id
JOIN members m ON m.member_id = $memberId
WHERE e.org_id = $orgId
  AND e.start_datetime >= NOW()
  AND e.status = 'active'
  AND (e.visibility = 'public'
    OR (e.visibility = 'members_only' AND m.role >= 'member'))
ORDER BY relevance_score DESC, e.start_datetime ASC
LIMIT 10;
```

7. Edge Cases & Error Handling

Edge Case	Detection Point	Resolution
Duplicate email registration	POST /auth/register	Return AUTH_004; offer "Sign In" or "Forgot Password"
Expired invitation link	GET /invitations/:code	INVITE_002 error; "Contact your admin for a new link"
Already-used single-use code	GET /invitations/:code	INVITE_001 error; offer manual org search
Org at max member capacity	POST /orgs/:id/join	Return ORG_001; show waitlist option if enabled
Admin rejects join request	PUT /admin/join-requests/:id	Member receives email with reason; status = APPROVAL_REJECTED
Network failure mid-wizard (web)	API call failure	Toast: "Connection issue — your progress is saved. Retrying..."; retry with exponential backoff (3x)
OAuth email differs from invite email	POST /auth/register/social	Use OAuth email; validate invite code is not locked to specific email
Browser/app closed mid-wizard	Page unload / app background	All entered data persisted to localStorage / AsyncStorage; wizard resumes at last incomplete step on return
No orgs in search results	GET /organizations?q=	Show: "Don't see your church? Ask your admin to send an invite link."
Approval notification email bounces	SES delivery webhook	Flag member email as undeliverable; admin dashboard shows warning
Push permission denied (mobile)	Expo Notifications.requestPermissionsAsync()	Gracefully skip device token registration; in-app notifications still work; nudge to enable in Settings

Edge Case	Detection Point	Resolution
Concurrent join requests (same email)	POST /orgs/:id/join	Database unique constraint on (member_id, org_id, status=PENDING); idempotent if re-submitted

8. Testing Strategy

8.1 Unit Tests (Vitest)

Unit tests cover isolated business logic. Target: $\geq 90\%$ coverage on all onboarding service methods.

Function / Unit	Key Test Cases
generateInviteCode()	Correct length; no ambiguous chars; uniqueness; collision-safe
validateInviteCode(code)	Valid code returns org/campus/role; expired code throws INVITE_002; used single-use throws INVITE_001; max-uses reached throws INVITE_003
hashPassword(plain)	bcrypt rounds = 12; hash length; deterministic comparison
calculateRelevanceScore(event, member)	Interest match = 2; same campus = 1; neither = 0; interest + campus = 2 (not additive)
buildEngagementSequence(member)	Day 1/3/7 messages present; Day 7 variant correct (group vs no-group); personalization tokens populated
onboardingStatusMachine(state, action)	Valid transitions only; PENDING cannot skip to COMPLETE; REJECTED can reapply
avatarProcessor(buffer)	Resize to 400×400; output JPEG; handles PNG/HEIC input; rejects > 10MB

8.2 Integration Tests (Supertest + PostgreSQL)

Integration tests run against a real PostgreSQL instance (Docker service in CI) with the full Prisma schema applied via `prisma migrate deploy`.

1. POST /auth/register → verifies member record created, JWT returned, welcome email enqueued in SQS mock
2. POST /auth/register (duplicate email) → verifies AUTH_004 returned, no duplicate record
3. POST /auth/register/social (Google OAuth mock) → verifies upsert behavior
4. GET /invitations/VALID123 → returns org, campus, role; GET /invitations/EXPIRED → INVITE_002
5. POST /organizations/:id/join (open_registration = true) → status immediately APPROVED
6. POST /organizations/:id/join (approval_required = true) → status PENDING, admin notification enqueued
7. PUT /admin/join-requests/:id (approve) → member status APPROVED, approval notification enqueued
8. Full registration → org join → profile → interests → serving → notifications happy path (7 sequential API calls)
9. Multi-tenancy: member from Org A cannot join Org B using Org A's invite code
10. Race condition: 100 concurrent joins to a single-use invite code → exactly 1 succeeds

8.3 End-to-End Tests (Playwright Web + Detox Mobile)

E2E tests run on the staging environment after each deployment. All flows must pass with < 2% flakiness.

Web E2E Scenarios (Playwright):

1. Self-registration → org search → campus selection → auto-approve → profile → interests → serving → notifications → tour → dashboard
2. Invitation link flow: arrive at /invite/CODE123 → org pre-filled → Step 1 account creation → auto-approve → complete wizard
3. Admin approval flow: member registers → admin logs in → approves request → member receives email → resumes onboarding
4. Skip all optional steps: click "Skip" on Steps 3, 4, 5 → dashboard shows "Complete your profile" nudge
5. Resume incomplete onboarding: complete Steps 1-3 → close browser → reopen → wizard resumes at Step 4
6. Error recovery: disconnect network at Step 5 → retry toast appears → reconnect → data saved correctly

Mobile E2E Scenarios (Detox):

1. Full happy path on iOS (iPhone 15 Pro) and Android (Pixel 7)
2. Push permission dialog interaction: Allow vs Deny → both paths complete correctly
3. Profile photo upload: pick from gallery → crop → upload → appears in header avatar
4. Calendar sync: tap "Add to Google Calendar" → deep link opens Google Calendar app

8.4 Accessibility Testing

Test Type	Tool	Acceptance Criteria
Automated scan	axe-core (CI)	Zero critical or serious WCAG 2.1 AA violations on all 7 screens
Keyboard navigation	Manual + Playwright	All form fields, buttons, and links reachable by Tab; Enter submits forms; Esc closes modals
Screen reader	VoiceOver (iOS/ macOS)	All inputs have aria-label; progress bar announces "Step N of 7"; error messages are live regions
Screen reader	TalkBack (Android)	Same as VoiceOver requirements; swipe navigation correct
Color contrast	Lighthouse + manual	All text 4.5:1 ratio; all interactive elements 3:1; tested across all 8 liturgical season themes
Dynamic Type (iOS)	Manual: size XXXL	No text truncation; layout does not break at maximum font scale

8.5 Performance Budgets

Metric	Target	Measurement Method
Onboarding bundle size (code-split)	< 120KB gzipped	webpack-bundle-analyzer in CI
Step transition time	< 400ms	Playwright Performance.mark() around step changes

Metric	Target	Measurement Method
Org search results latency	< 500ms p95	CloudWatch API latency on GET /organizations
Invitation code validation	< 200ms p95	CloudWatch API latency on GET / invitations/:code
Avatar upload (< 5MB image)	< 3s end-to-end	Playwright navigation timing on Step 3
Mobile wizard cold start	< 1.5s	Detox app launch + first render timing
Welcome email delivery	< 60s from registration	SES delivery webhook + integration test assertion

8.6 Definition of Done (for Onboarding Feature)

1. All 7 web screens render correctly in Chrome, Firefox, and Safari (latest 2 versions)
2. Mobile wizard completes successfully on iOS 16+ and Android 12+
3. All 10 user stories have passing Playwright E2E tests
4. Unit test coverage $\geq 90\%$ for onboarding service files
5. Zero axe-core critical/serious violations on all wizard steps
6. Invitation code validation handles all error states (expired, used, not-found)
7. Admin approval flow sends notifications within 2 minutes (staging verified)
8. Onboarding wizard state persists correctly through browser close + reopen
9. First-week engagement sequence fires on a test member in staging environment
10. All onboarding analytics events fire correctly
11. Feature flag (FEATURE_NEW_ONBOARDING) works: old flow unchanged when flag is off
12. Security: RBAC prevents unauthenticated access to any member data endpoint; multi-tenant isolation verified

9. Implementation Sprint Plan

The New Member Onboarding feature maps to the following implementation tasks. These are additive to the main ROCA Sprint 1 foundation work (which must be completed first).

#	Task	Sprint	Acceptance Criterion
OB-1	DB migrations (new columns + 3 new tables)	Sprint 1	All migrations apply cleanly; invitation_codes, member_interests, notification_preferences tables exist
OB-2	Invitation code generation & validation API	Sprint 1	POST /admin/invitations creates codes; GET / invitations/:code validates; all error states handled
OB-3	Registration & OAuth endpoints	Sprint 1	Email + social auth work; duplicate email caught; welcome email queued
OB-4	Org join request + admin approval endpoints	Sprint 1	Join $\rightarrow$ PENDING flow and auto-approve flow both work; admin notified
OB-5	Member profile, interests, serving, notif. prefs APIs	Sprint 2	All 5 PUT endpoints persist data; onboarding step tracking works
OB-6	Web wizard (Steps 1–7) React implementation	Sprint 2	All 7 steps render; Zustand state persists; progress bar accurate; skip/resume work
OB-7	Interest-to-event personalization (dashboard feed)	Sprint 2	Dashboard Upcoming Events shows interest-matched events first

#	Task	Sprint	Acceptance Criterion
OB-8	Mobile onboarding wizard (React Native)	Sprint 5	Full flow works on iOS + Android; push permission prompt fires at Step 6
OB-9	First-week engagement automation (Lambda + SQS)	Sprint 6	Day 1/3/7 messages send with correct personalization; Day 7 variant logic correct
OB-10	E2E test suite for all onboarding flows	Sprint 6	6 Playwright scenarios + 4 Detox scenarios passing in CI; < 2% flakiness

10. Appendix

10.1 Onboarding Status Values Reference

Status Value	Meaning
REGISTERED	Account created; no org association yet
PENDING_APPROVAL	Join request submitted; awaiting admin review
APPROVAL_REJECTED	Admin rejected join request; email sent with reason
APPROVED	Admin approved or auto-approved; wizard continues from Step 3
ONBOARDING_INCOMPLETE	Steps 3–7 partially skipped; member is active but nudge shown
ONBOARDING_COMPLETE	All wizard steps completed; full member experience; engagement sequence starts

10.2 Analytics Events

All analytics events use the ROCA telemetry system (Segment or Amplitude). Events are fired client-side with the member_id, org_id, and timestamp as standard properties.

Event Name	Properties
onboarding_started	entry_type: invite search; invite_code: string null
onboarding_step_viewed	step: 1-7
onboarding_step_completed	step: 1-7; duration_ms: number
onboarding_step_skipped	step: 1-7; reason: user_action
onboarding_org_joined	org_id: string; campus_id: string; auto_approved: boolean
onboarding_completed	total_duration_ms: number; steps_skipped: number[]
onboarding_abandoned	last_step: 1-7; session_duration_ms: number
invite_code_redeemed	code_id: string; org_id: string; role: string
groups_joined_during_onboarding	group_ids: string[]; count: number
push_permission_granted	platform: ios android
push_permission_denied	platform: ios android
calendar_sync_initiated	calendar_type: google apple ical

10.3 Error Codes Reference

Error Code	HTTP	Message
------------	------	---------

Error Code	HTTP	Message
AUTH_004	409	Email already registered for this organization
AUTH_005	401	Email verification required before proceeding
INVITE_001	404	Invitation code not found
INVITE_002	410	Invitation code has expired
INVITE_003	409	Invitation code has reached its maximum use limit
ORG_001	409	Organization has reached maximum member capacity
ORG_002	409	Member already has a pending or active join request for this org
ONBOARD_001	400	Onboarding step submitted out of order

10.4 Glossary

Term	Definition
EARS	Easy Approach to Requirements Syntax; a structured format for writing acceptance criteria using WHEN/THE SYSTEM SHALL patterns
Invite Code	An 8-character alphanumeric token generated by an admin that pre-fills org context and optionally grants auto-approval and role assignment
Onboarding Wizard	The 7-step progressive web/mobile flow that guides new members from account creation through feature tour
Open Registration	An org configuration flag (open_registration = true) that auto-approves all join requests without admin review
RRULE	RFC 5545 recurrence rule format used to define recurring event patterns (e.g., FREQ=WEEKLY;BYDAY=SU)
JWT	JSON Web Token; the signed token issued on login, valid for 15 minutes, used to authenticate all API requests
Multi-tenancy	Architecture pattern where multiple organizations share the same platform but data is strictly isolated by org_id
iCal / .ics	Internet Calendaring standard (RFC 5545); used for calendar export and subscription URLs

10.5 Revision History

Version	Date	Author	Summary
1.0	2026-08-26	Design Team	Initial release based on approved outline; all 7 screens, 10 user stories, full architecture